EJERCICIOS DE PROGRAMACIÓN

Orientada a Objetos en Java

30 Ejercicios Prácticos

Clases · Objetos · ArrayList · Herencia

Dificultad progresiva: Fácil → Medio → Difícil

10	10	10	30
Fácil	**Medio**	**Difícil**	**Total**

NIVEL 1 — FÁCIL

Ejercicios 1 al 10

#1 Clase Persona

Conceptos: Clase | Objeto

Enunciado:

Crea la clase Persona con atributos privados nombre, edad y correo. Agrega un constructor con los tres atributos, métodos getters y setters con validación, y un método presentarse() que imprima sus datos formateados en consola.

Métodos sugeridos:

- `Persona(String nombre, int edad, String correo)`
- `getNombre() / setNombre()`
- `getEdad() / setEdad()`
- `getCorreo() / setCorreo()`
- `presentarse()`

Pista de implementacion:
Declara los atributos como private. El setter de edad debe validar que sea positiva. El método presentarse() puede usar System.out.printf(). En el main() instancia al menos 2 objetos distintos.

Tiempo estimado: 20-30 min	**[ ] Completado**

#2 Clase Automóvil

Conceptos: Clase | Objeto

Enunciado:

Define la clase Automovil con atributos marca, modelo, año y velocidadActual (inicia en 0). Implementa los métodos acelerar(int km) que sume km a la velocidad, frenar(int km) que reste validando que no sea negativa, y getEstado() que muestre todos los datos.

Métodos sugeridos:

- `Automovil(String marca, String modelo, int anio)`
- `acelerar(int km)`
- `frenar(int km)`
- `getEstado()`

Pista de implementacion:
En frenar() usa: if(velocidadActual - km < 0) velocidadActual = 0; else velocidadActual -= km. En el main() simula una secuencia: acelerar, frenar, mostrar estado.

Tiempo estimado: 20-30 min	**[ ] Completado**

#3 Clase Rectángulo

Conceptos: Clase | Objeto

Enunciado:

Construye la clase Rectangulo con atributos double base y double altura. Agrega métodos calcularArea() que retorne base × altura, calcularPerimetro() que retorne 2 × (base + altura), y esCuadrado() que retorne true si base == altura.

Métodos sugeridos:

- `Rectangulo(double base, double altura)`
- `calcularArea() → double`
- `calcularPerimetro() → double`
- `esCuadrado() → boolean`

Pista de implementacion:
Usa double para los atributos para permitir decimales. esCuadrado() retorna: return base == altura. Prueba con varios rectángulos, incluyendo uno cuadrado.

Tiempo estimado: 20-30 min	**[ ] Completado**

#4 Clase Estudiante con Calificación

Conceptos: Clase | Objeto

Enunciado:

Crea la clase Estudiante con nombre, código y nota (double de 0.0 a 10.0). El setter de nota debe validar el rango. Agrega el método getCalificacion() que retorne 'Aprobado' si nota >= 6.0, o 'Reprobado' si es menor. También agrega toString() que muestre todos los datos.

Métodos sugeridos:

- `Estudiante(String nombre, String codigo, double nota)`
- `getNota() / setNota() con validación`
- `getCalificacion() → String`
- `toString()`

Pista de implementacion:
Usa un operador ternario en getCalificacion(): return nota >= 6.0 ? 'Aprobado' : 'Reprobado'. En setNota() verifica: if(nota >= 0 && nota <= 10) this.nota = nota.

Tiempo estimado: 20-30 min	**[ ] Completado**

#5 Clase Producto con IVA

Conceptos: Clase | Objeto

Enunciado:

Modela un Producto con nombre, precio (double) y categoría (String). Agrega getPrecioConIVA() que sume el 19% al precio base. Agrega aplicarDescuento(double pct) que reduzca el precio base en ese porcentaje. Imprime el precio antes y después del descuento.

Métodos sugeridos:

- `Producto(String nombre, double precio, String categoria)`
- `getPrecioConIVA() → double`
- `aplicarDescuento(double pct)`
- `toString()`

Pista de implementacion:
*getPrecioConIVA(): return precio * 1.19. aplicarDescuento(10) debe reducir el precio en un 10%: precio = precio * (1 - pct/100). Valida que pct esté entre 0 y 100.*

Tiempo estimado: 20-30 min	**[ ] Completado**

#6 Cuenta Bancaria

Conceptos: Clase | Objeto

Enunciado:

Crea la clase CuentaBancaria con titular, numeroCuenta y saldo (double). Implementa depositar(double monto) que sume al saldo validando monto positivo, retirar(double monto) validando fondos suficientes, y getEstado() que imprima todos los datos formateados.

Métodos sugeridos:

- `CuentaBancaria(String titular, String numero, double saldoInicial)`
- `depositar(double monto)`
- `retirar(double monto)`
- `getEstado()`

Pista de implementacion:
En retirar(): if(monto > 0 && monto <= saldo) saldo -= monto; else System.out.println('Fondos insuficientes'). Usa String.format('$%,.2f', saldo) para mostrar el saldo.

Tiempo estimado: 20-30 min	**[ ] Completado**

#7 Lista de Productos

Conceptos: ArrayList

Enunciado:

Crea la clase Tienda que contenga un ArrayList<Producto> (reutiliza la clase del ejercicio 5). Agrega los métodos: agregarProducto(Producto p), eliminarPorNombre(String nombre), listarTodos() y buscarPorCategoria(String cat) que retorne la lista filtrada.

Métodos sugeridos:

- `agregarProducto(Producto p)`
- `eliminarPorNombre(String nombre)`

- » `listarTodos()`
- » `buscarPorCategoria(String cat) → ArrayList<Producto>`

Pista de implementacion:
Para eliminar usa: lista.removeIf(p -> p.getNombre().equalsIgnoreCase(nombre)). buscarPorCategoria() recorre la lista y agrega a un nuevo ArrayList los que coincidan.

Tiempo estimado: 20-30 min	**[ ] Completado**

#8 ArrayList de Estudiantes

Conceptos: ArrayList

Enunciado:

En el método main(), crea un ArrayList<Estudiante> y agrega 5 estudiantes con distintas notas. Luego: (a) imprime solo los aprobados (nota >= 6.0), (b) calcula e imprime el promedio de la lista, (c) encuentra e imprime al estudiante con la nota más alta.

Métodos sugeridos:

- » `Crear ArrayList<Estudiante>`
- » `Filtrar aprobados con for-each`
- » `Calcular promedio`
- » `Encontrar el de mayor nota`

Pista de implementacion:
Para filtrar aprobados usa un for-each con un if. Para el promedio suma todas las notas y divide entre el tamaño. Para el mejor: recorre guardando el de mayor nota en una variable auxiliar.

Tiempo estimado: 20-30 min	**[ ] Completado**

#9 Vehículo → Auto / Moto

Conceptos: Herencia

Enunciado:

Crea la clase base Vehiculo con atributos marca (String) y velocidadMax (int), y el método describir(). Crea la subclase Auto que agregue numPuertas (int), y la subclase Moto que agregue tipo (String: 'deportiva' o 'urbana'). Sobreescribe describir() en ambas usando super.

Métodos sugeridos:

- » `Vehiculo(String marca, int velocidadMax)`
- » `describir() en Vehiculo`
- » `Auto(marca, velMax, numPuertas) — sobreescribe describir()`
- » `Moto(marca, velMax, tipo) — sobreescribe describir()`

Pista de implementacion:
En Auto y Moto, el constructor llama super(marca, velocidadMax) primero, luego asigna sus propios atributos. En describir() llama super.describir() y luego imprime los atributos propios.

Tiempo estimado: 20-30 min	**[ ] Completado**

#10 Figura → Círculo / Triángulo

Conceptos: **Herencia** | **Clase**

Enunciado:

Define la clase abstracta Figura con el atributo color (String) y el método abstract double calcularArea(). Implementa la subclase Circulo con atributo radio, y Triangulo con base y altura. En el main() crea un array Figura[] con ambos tipos y llama calcularArea() sobre cada uno.

Métodos sugeridos:

- `abstract class Figura — abstract calcularArea()`
- `Circulo extends Figura — calcularArea()`
- `Triangulo extends Figura — calcularArea()`
- `Figura[] figuras = {...} en main()`

Pista de implementacion:
*En Circulo: return Math.PI * radio * radio. En Triangulo: return (base * altura) / 2. El array Figura[] permite guardar objetos de distintos tipos gracias al polimorfismo.*

Tiempo estimado: 20-30 min	**[ ] Completado**

NIVEL 2 — MEDIO

Ejercicios 11 al 20

#11 Biblioteca de Libros

Conceptos: Clase | ArrayList

Enunciado:

Crea la clase Libro con titulo, autor, isbn y precio. Crea la clase Biblioteca con ArrayList<Libro>. Implementa: agregar(Libro), eliminar(String isbn), buscarPorAutor(String autor) que retorne una lista, libroMasCaro() que retorne el libro de mayor precio, y listarTodos().

Métodos sugeridos:

- `agregar(Libro l)`
- `eliminar(String isbn)`
- `buscarPorAutor(String autor) → ArrayList<Libro>`
- `libroMasCaro() → Libro`
- `listarTodos()`

Pista de implementacion:
buscarPorAutor() recorre la lista y agrega al resultado los que coincidan con autor.equalsIgnoreCase(). Para libroMasCaro(), recorre guardando el máximo en una variable Libro max = null.

Tiempo estimado: 30-45 min	**[ ] Completado**

#12 Empleado → Fijo / Temporal

Conceptos: Herencia

Enunciado:

Define la clase abstracta Empleado con nombre (String) e id (String), y el método abstract double calcularPago(). Crea EmpleadoFijo con salarioMensual, y EmpleadoTemporal con horasTrabajadas y tarifaHora. En el main() crea un ArrayList<Empleado> con ambos tipos y calcula la nómina total.

Métodos sugeridos:

- `abstract class Empleado — abstract calcularPago()`
- `EmpleadoFijo.calcularPago() → salarioMensual`
- `EmpleadoTemporal.calcularPago() → horas * tarifa`
- `ArrayList<Empleado> con cálculo de nómina total`

Pista de implementacion:
*EmpleadoFijo.calcularPago() retorna salarioMensual. EmpleadoTemporal.calcularPago() retorna horasTrabajadas * tarifaHora. Para la nómina: suma todos los calcularPago() del ArrayList.*

Tiempo estimado: 30-45 min	**[ ] Completado**

#13 Clase Matriz 2D

Conceptos: Clase | Objeto

Enunciado:

Crea la clase Matriz con un array interno double[][] datos. El constructor recibe filas y columnas. Implementa: setValor(int fila, int col, double val), getValor(int fila, int col), calcularSuma() que sume todos los elementos, e imprimirMatriz() que la muestre formateada con printf.

Métodos sugeridos:

- `Matriz(int filas, int cols)`
- `setValor(int f, int c, double val)`
- `getValor(int f, int c) → double`
- `calcularSuma() → double`
- `imprimirMatriz()`

Pista de implementacion:
El constructor: this.datos = new double[filas][cols]. Para imprimirMatriz() usa dos for anidados: System.out.printf('%8.2f', datos[i][j]). Al terminar cada fila imprime un salto de línea.

Tiempo estimado: 30-45 min	**[ ] Completado**

#14 Ranking con Ordenamiento

Conceptos: ArrayList

Enunciado:

Crea la clase Jugador con nombre (String) y puntos (int). Crea la clase Ranking con ArrayList<Jugador> y los métodos: agregar(Jugador), clasificar() que ordene de mayor a menor puntos, mostrarTop(int n) que muestre los primeros n jugadores, y puntosTotales().

Métodos sugeridos:

- `agregar(Jugador j)`
- `clasificar() — ordena de mayor a menor`
- `mostrarTop(int n)`
- `puntosTotales() → int`

Pista de implementacion:
Para ordenar: lista.sort(Comparator.comparingInt(Jugador::getPuntos).reversed()). mostrarTop(3) puede usar un bucle hasta Math.min(n, lista.size()). Asegúrate de llamar clasificar() antes de mostrarTop().

Tiempo estimado: 30-45 min	**[ ] Completado**

#15 Animal con Polimorfismo

Conceptos: Herencia

Enunciado:

Crea la clase base Animal con nombre (String), edad (int) y el método hacerSonido() que imprima un sonido genérico. Crea las subclases Perro, Gato y Vaca, cada una sobreescribiendo hacerSonido(). En el main() crea un ArrayList<Animal> con los 3 tipos y llama hacerSonido() en todos.

Métodos sugeridos:

- » `Animal — hacerSonido() genérico + comer()`
- » `Perro — hacerSonido(): 'Guau guau'`
- » `Gato — hacerSonido(): 'Miau miau'`
- » `Vaca — hacerSonido(): 'Muuu'`
- » `ArrayList<Animal> con loop polimórfico`

Pista de implementacion:
Este ejercicio muestra polimorfismo: la referencia es Animal pero el método ejecutado depende del objeto real. Agrega también comer() en Animal con un mensaje que incluya el nombre del animal.

Tiempo estimado: 30-45 min	**[ ] Completado**

#16 Gestor de Notas Estadístico

Conceptos: ArrayList

Enunciado:

Crea la clase GestorNotas con ArrayList<Double>. Implementa: agregarNota(double), eliminarNota(double), calcularPromedio(), obtenerMaxima(), obtenerMinima(), cuantasAprobaron() que cuente las notas >= 6.0, y mostrarResumen() que imprima todas las estadísticas.

Métodos sugeridos:

- » `agregarNota(double n) / eliminarNota(double n)`
- » `calcularPromedio() → double`
- » `obtenerMaxima() / obtenerMinima() → double`
- » `cuantasAprobaron() → int`
- » `mostrarResumen()`

Pista de implementacion:
Para promedio suma con un for-each y divide entre lista.size(). Usa Collections.max(lista) y Collections.min(lista). cuantasAprobaron(): recorre con un for-each y cuenta las que cumplen la condición.

Tiempo estimado: 30-45 min	**[ ] Completado**

#17 Clase Pila (Stack Manual)

Conceptos: Clase | Objeto

Enunciado:

Implementa la clase Pila usando un ArrayList internamente. Agrega los métodos: push(Object elem) que agregue al final, pop() que retorne y elimine el último elemento, peek() que solo lea el último sin eliminarlo, isEmpty() y size(). Valida que la pila no esté vacía antes de pop() y peek().

Métodos sugeridos:

- `push(Object elem)`
- `pop() → Object (lanza excepción si vacía)`
- `peek() → Object (lanza excepción si vacía)`
- `isEmpty() → boolean`
- `size() → int`

Pista de implementacion:
push(): lista.add(elem). pop(): return lista.remove(lista.size()-1). peek(): return lista.get(lista.size()-1). Lanza IllegalStateException si la pila está vacía en pop() o peek().

Tiempo estimado: 30-45 min	**[ ] Completado**

#18 Cuenta → Ahorros / Corriente

Conceptos: Herencia

Enunciado:

Clase base Cuenta con numeroCuenta y saldo. Métodos depositar() y retirar(). Subclase CuentaAhorros agrega tasaInteres y el método aplicarIntereses() que suma al saldo. Subclase CuentaCorriente agrega cupoSobregiro y sobreescribe retirar() para permitir saldo negativo hasta el cupo.

Métodos sugeridos:

- `Cuenta — depositar(), retirar(), getEstado()`
- `CuentaAhorros — tasaInteres, aplicarIntereses()`
- `CuentaCorriente — cupoSobregiro, sobreescribe retirar()`

Pista de implementacion:
*CuentaAhorros.aplicarIntereses(): saldo += saldo * tasaInteres. En CuentaCorriente.retirar(): permitir si monto <= saldo + cupoSobregiro. Sobreescribe getEstado() en cada subclase para mostrar sus datos extra.*

Tiempo estimado: 30-45 min	**[ ] Completado**

#19 Inventario con Búsqueda Avanzada

Conceptos: ArrayList | Clase

Enunciado:

Crea Producto con nombre, categoria, precio y stock. Clase Inventario con ArrayList<Producto>. Implementa: buscarPorCategoria(String cat), filtrarPorRangoPrecio(double min, double max), productosAgotados() que retorne los de stock == 0, y valorTotalInventario() que sume precio × stock de todos.

Métodos sugeridos:

- `buscarPorCategoria(String cat) → ArrayList<Producto>`
- `filtrarPorRangoPrecio(double min, double max) → ArrayList<Producto>`
- `productosAgotados() → ArrayList<Producto>`
- `valorTotalInventario() → double`

Pista de implementacion:
*Todos los métodos de búsqueda retornan ArrayList<Producto>. valorTotalInventario(): double total = 0; for(Producto p : lista) total += p.getPrecio() * p.getStock(); return total.*

Tiempo estimado: 30-45 min	**[ ] Completado**

#20 Sólidos 3D

Conceptos: Herencia

Enunciado:

Clase abstracta Solido3D con color (String) y los métodos abstractos calcularVolumen() y calcularAreaSuperficial(). Implementa las subclases: Esfera (radio), Cubo (lado) y Cilindro (radio, altura). En el main() crea una lista de sólidos y muestra volumen y área de cada uno.

Métodos sugeridos:

- `abstract class Solido3D — abstract calcularVolumen(), calcularAreaSuperficial()`
- `Esfera — implementa ambos métodos`
- `Cubo — implementa ambos métodos`
- `Cilindro — implementa ambos métodos`

Pista de implementacion:
*Esfera: volumen = (4.0/3)*PI*r^3, área = 4*PI*r^2. Cubo: volumen = $lado^3$, área = 6*$lado^2$. Cilindro: volumen = PI*r^2*h, área = 2*PI*r*(r+h). Usa Math.PI y Math.pow().*

Tiempo estimado: 30-45 min	**[ ] Completado**

NIVEL 3 — DIFÍCIL

Ejercicios 21 al 30

#21 Lista con Capacidad Máxima

Conceptos: Clase | ArrayList

Enunciado:

Crea la clase ListaLimitada que use un ArrayList internamente pero tenga una capacidad máxima. Crea la excepción personalizada CapacidadExcedidaException. Si se intenta agregar un elemento cuando la lista está llena, lanza la excepción. Agrega también los métodos remover, obtener, tamaño y listar.

Métodos sugeridos:

- » `class CapacidadExcedidaException extends RuntimeException`
- » `agregar(Object elem) — lanza excepción si llena`
- » `remover(int indice) → Object`
- » `obtener(int indice) → Object`
- » `tamaño() → int`

Pista de implementacion:
class CapacidadExcedidaException extends RuntimeException { public CapacidadExcedidaException(String msg){ super(msg); } }. En agregar(): if(lista.size() >= capacidadMax) throw new CapacidadExcedidaException('Lista llena, capacidad: '+capacidadMax).

Tiempo estimado: 45-60 min	**[ ] Completado**

#22 Herencia Multinivel: Ser → Animal → Mamifero → Perro

Conceptos: Herencia

Enunciado:

Construye una jerarquía de 4 niveles. Clase Ser con el método respirar(). Clase Animal extiende Ser y agrega moverse(). Clase Mamifero extiende Animal y agrega amamantar(). Clase Perro extiende Mamifero y agrega ladrar(). Demuestra en el main() que Perro puede invocar todos los métodos heredados.

Métodos sugeridos:

- » `Ser — respirar()`
- » `Animal extends Ser — moverse()`
- » `Mamifero extends Animal — amamantar()`
- » `Perro extends Mamifero — ladrar()`
- » `Demostrar herencia acumulada en main()`

Pista de implementacion:
Cada clase llama super() en el constructor para pasar atributos hacia arriba. Un objeto Perro hereda todos los métodos de la cadena: p.respirar(), p.moverse(), p.amamantar(), p.ladrar() — todos son accesibles.

Tiempo estimado: 45-60 min	**[ ] Completado**

#23 Sistema de Reservas con Enum

Conceptos: ArrayList | Clase

Enunciado:

Crea el enum EstadoReserva {PENDIENTE, CONFIRMADA, CANCELADA}. Clase Reserva con id, cliente, fecha, habitacion y estado. Clase CentralReservas con ArrayList<Reserva> y los métodos: agregar, confirmar(int id), cancelar(int id), buscarPorCliente(String cliente) y reservasDelDia(String fecha).

Métodos sugeridos:

- `enum EstadoReserva`
- `Reserva con todos sus atributos`
- `confirmar(int id) / cancelar(int id)`
- `buscarPorCliente(String cliente) → ArrayList<Reserva>`
- `reservasDelDia(String fecha) → ArrayList<Reserva>`

Pista de implementacion:
confirmar(id) recorre la lista buscando la reserva con ese id y cambia su estado a CONFIRMADA. reservasDelDia(fecha) filtra las que tengan la misma fecha. Usa equals() para comparar fechas como String.

Tiempo estimado: 45-60 min	**[ ] Completado**

#24 Interfaces Múltiples: Volador y Nadador

Conceptos: Herencia | Clase

Enunciado:

Crea las interfaces Volador con volar() y Nadador con nadar(). Clase base Animal con nombre y comer(). Clase Pato que extiende Animal e implementa Volador y Nadador. Clase Aguila que extiende Animal e implementa solo Volador. Crea un ArrayList<Volador> con ambos y llama volar() en todos.

Métodos sugeridos:

- `interface Volador — volar()`
- `interface Nadador — nadar()`
- `Pato extends Animal implements Volador, Nadador`
- `Aguila extends Animal implements Volador`
- `ArrayList<Volador> — polimorfismo de interface`

Pista de implementacion:
Pato debe implementar volar() y nadar() obligatoriamente. Aguila solo implementa volar(). ArrayList<Volador> puede contener objetos Pato y Aguila porque ambos implementan Volador. Prueba también: Volador v = new Pato().

Tiempo estimado: 45-60 min	**[ ] Completado**

#25 Clase Cola (Queue Manual)

Conceptos: Clase | ArrayList

Enunciado:

Implementa la clase Cola usando un ArrayList internamente. Métodos: encolar(Object elem) que agregue al final, desencolar() que retorne y elimine el primero (FIFO), frente() que solo lea el primero, estaVacia(), tamaño() e imprimirCola(). Simula una fila de banco con 5 clientes.

Métodos sugeridos:

- `encolar(Object elem)`
- `desencolar() → Object (FIFO, lanza excepción si vacía)`
- `frente() → Object`
- `estaVacia() → boolean`
- `tamaño() → int`
- `imprimirCola()`

Pista de implementacion:
encolar(): lista.add(elem). desencolar(): return lista.remove(0) — elimina el de índice 0 (el primero en entrar). frente(): return lista.get(0). Lanza excepción si la cola está vacía al desencolar o leer el frente.

Tiempo estimado: 45-60 min	**[ ] Completado**

#26 Vehículo con Patrón Factory

Conceptos: Herencia | Clase

Enunciado:

Crea la jerarquía Vehiculo → Auto, Moto, Camion, cada una con atributos propios y el método describir(). Crea la clase VehiculoFactory con el método estático crear(String tipo, String marca) que retorne el objeto correcto usando switch. En el main() usa la fábrica para crear un ArrayList<Vehiculo>.

Métodos sugeridos:

- `abstract class Vehiculo — describir()`
- `Auto, Moto, Camion — sobreescriben describir()`
- `VehiculoFactory.crear(String tipo, String marca) → Vehiculo`
- `ArrayList<Vehiculo> usando la fábrica`

Pista de implementacion:
public static Vehiculo crear(String tipo, String marca) { return switch(tipo.toLowerCase()) { case 'auto' -> new Auto(marca, ...); case 'moto' -> new Moto(marca, ...); default -> throw new IllegalArgumentException('Tipo desconocido: '+tipo); }; }

Tiempo estimado: 45-60 min	**[ ] Completado**

#27 Sistema de Calificaciones Estadístico

Conceptos: ArrayList | Clase

Enunciado:

Clase Curso con nombre y ArrayList<Estudiante>. Métodos: promedioCurso(), desviacionEstandar(), estudiantesMayoresQuePromedio() que retorne la lista filtrada, y distribucion() que cuente cuántos estudiantes hay en cada rango: 0-4 (Bajo), 5-6 (Básico), 7-8 (Alto), 9-10 (Superior).

Métodos sugeridos:

- `promedioCurso() → double`
- `desviacionEstandar() → double`
- `estudiantesMayoresQuePromedio() → ArrayList<Estudiante>`
- `distribucion() — imprime conteo por rango`

Pista de implementacion:

Desviación estándar: (1) calcula el promedio, (2) suma (nota-promedio)² de cada uno, (3) divide entre n, (4) aplica Math.sqrt(). distribucion() usa 4 contadores int y un if-else en cadena por rango.

Tiempo estimado: 45-60 min	**[ ] Completado**

#28 Sobreescritura en Cadena con super

Conceptos: Herencia

Enunciado:

Clase Vehiculo con descripcion() que imprime marca y año. Clase Auto sobreescribe descripcion(), llama super.descripcion() al inicio y agrega numPuertas. Clase AutoDeportivo extiende Auto, sobreescribe descripcion(), llama super.descripcion() y agrega velocidadMax y tiempoAceleracion.

Métodos sugeridos:

- `Vehiculo — descripcion(): imprime marca y año`
- `Auto — @Override descripcion(): llama super + imprime numPuertas`
- `AutoDeportivo — @Override descripcion(): llama super + imprime velocidadMax y aceleracion`

Pista de implementacion:

Cada nivel agrega su información a la del nivel anterior: AutoDeportivo muestra marca + año (de Vehiculo), numPuertas (de Auto) y velocidadMax + tiempoAceleracion (propio). Así se acumula la descripción completa con super.

Tiempo estimado: 45-60 min	**[ ] Completado**

#29 Agenda de Contactos Completa

Conceptos: ArrayList | Clase

Enunciado:

Enum TipoContacto {PERSONAL, TRABAJO, FAMILIA}. Clase Contacto con nombre, telefono, email y tipo. Clase Agenda con ArrayList<Contacto>. Métodos: agregar, eliminar, buscarPorNombre(String texto) que busque por coincidencia parcial, agruparPorTipo() que retorne un HashMap y exportar() que imprima todos formateados.

Métodos sugeridos:

- » `enum TipoContacto`
- » `buscarPorNombre(String texto) → ArrayList<Contacto> (búsqueda parcial)`
- » `agruparPorTipo() → HashMap<TipoContacto, ArrayList<Contacto>>`
- » `exportar() — imprime todos formateados`

Pista de implementacion:
buscarPorNombre(texto): filtra con nombre.toLowerCase().contains(texto.toLowerCase()). agruparPorTipo(): crea un HashMap<TipoContacto, ArrayList<Contacto>>() y recorre la lista asignando cada contacto a su grupo.

Tiempo estimado: 45-60 min	**[ ] Completado**

#30 Mini Sistema Hospitalario

Conceptos: **Herencia** | **ArrayList** | **Clase**

Enunciado:

Clase base Persona con nombre y cedula. Subclase Paciente agrega historialClinico (ArrayList<String>) y enfermedad. Subclase Doctor agrega especialidad y ArrayList<Paciente> pacientesACargo. Clase Hospital con ArrayList<Doctor> y ArrayList<Paciente>. Métodos: asignarPaciente(Doctor, Paciente), buscarPaciente(String nombre) y reporteGeneral().

Métodos sugeridos:

- » `Persona(nombre, cedula)`
- » `Paciente extends Persona — historialClinico, enfermedad`
- » `Doctor extends Persona — especialidad, ArrayList<Paciente>`
- » `Hospital — asignarPaciente(), buscarPaciente(), reporteGeneral()`

Pista de implementacion:
asignarPaciente() agrega el Paciente al doctor.pacientesACargo y registra en historialClinico del paciente: 'Asignado al Dr. X - especialidad'. reporteGeneral() muestra por cada doctor cuántos pacientes tiene y los lista.

Tiempo estimado: 45-60 min	**[ ] Completado**